

Arduino IDE - LoRa V1.2

Date: 05/12/2025

Autor: Luís Costa

Indices

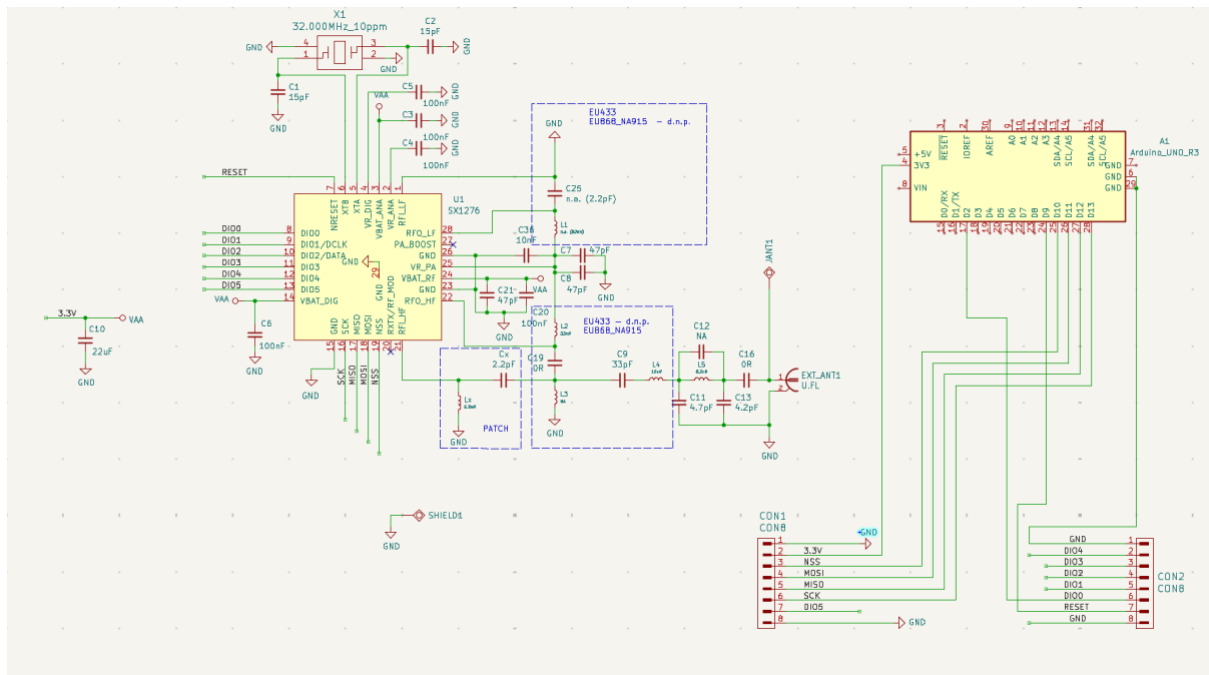
1.	Introdução ao Sistema	2
2.	Diagrama de Ligação de Hardware (Interface SPI)	2
3.	Implementação de Software (Código Arduino IDE para teste do módulo)	2
3.1.	Código Comum (Setup)	2
3.2.	Código do Transmissor (TX)	3
3.3.	Código do Recetor (RX)	4
3.4.	Código para teste individual do módulo LoRa com o arduino	5
4.	Plano de Desenvolvimento e Testes do Módulo de Comunicação LoRa	6
4.1.	Adaptação de Software e Integração de Hardware	6
4.1.1.	Transição da Plataforma Host (Arduino Uno → ESP32-S3)	6
4.2.	Otimização dos Parâmetros de Rádio (Spread Factor)	7
4.2.1.	Resultados da Otimização dos Parâmetros de Rádio (Spread Factor)	7
4.2.1.1.	Códigos de Diagnóstico e Telemetria	9
4.3.	Definição da Formatação da Mensagem	14
4.3.1.	Conteúdo do Payload e Estrutura	14
4.3.2.	Estrutura da mensagem para teste do SpreadFactor no arduino	15
4.3.3.	Implementação de Segurança e Integridade	15
5.	Bibliografia	16
6.	Histórico de versões	16

1. Introdução ao Sistema

Este documento estabelece o protocolo de comunicação SPI para o Módulo OLIMEX LoRa868 (chip Semtech SX1276) utilizando o Arduino Uno como microcontrolador host. O módulo opera na frequência de 868 MHz (Brequência ISM Europeia).

2. Diagrama de Ligação de Hardware (Interface SPI)

A comunicação SPI é a interface nativa do chip SX1276. Devido à diferença de voltagem entre o Arduino Uno (5V) e o LoRa868 (3.3V), é fundamental o uso de um Conversor de Nível Lógico (Level Shifter).



3. Implementação de Software (Código Arduino IDE para teste do módulo)

A biblioteca LoRa.h é necessária para gerir o transmissor via SPI.

3.1. Código Comum (Setup)

```
#include <LoRa.h>
```

```

// (Ajustar conforme as ligações feitas no Arduino)

#define SS_PIN 10
#define RST_PIN 9
#define DI0_PIN 2

const long FREQUENCY = 868E6; // 868 MHz

void setup() {
  Serial.begin(9600);
  while (!Serial);
  Serial.println("Inicializando LoRa...");

  // Configurar Pinos e Iniciar Módulo na Frequência
  LoRa.setPins(SS_PIN, RST_PIN, DI0_PIN);
  if (!LoRa.begin(FREQUENCY)) {
    Serial.println("ERRO: Falha na inicialização do LoRa!");
    while (1);
  }

  // Definir a Potência de Transmissão (Máx. do LoRa868)
  LoRa.setTxPower(14);
  Serial.println("Módulo LoRa pronto.");
}

```

3.2. Código do Transmissor (TX)

```

int counter = 0;

void loop() {
  String message = "Pacote de Teste #" + String(counter);
  Serial.print("A enviar: ");
  Serial.println(message);

  // Sequência de Transmissão SPI:
  LoRa.beginPacket(); // Inicia o pacote
  LoRa.print(message); // Escreve os dados
}

```

```
LoRa.endPacket(); // Envia o pacote

counter++;
delay(5000);
}
```

3.3. Código do Recetor (RX)

```
void loop() {
    // A função parsePacket() verifica o pino DIO0 para sinal de receção.
    int packetSize = LoRa.parsePacket();

    if (packetSize) {
        Serial.print("Pacote Recebido: ");

        // Ler dados via SPI
        while (LoRa.available()) {
            Serial.print((char)LoRa.read());
        }

        // Leitura de Diagnóstico (Qualidade do Sinal)
        long rssi = LoRa.packetRssi();
        float snr = LoRa.packetSnr();

        Serial.println();
        Serial.print(" -> RSSI: ");
        Serial.print(rssi);
        Serial.println(" dBm");
        Serial.print(" -> SNR: ");
        Serial.print(snr);
        Serial.println(" dB");
    }
}
```

3.4. Código para teste individual do módulo LoRa com o arduino

```
#include <SPI.h>
#include <LoRa.h>

// Definição dos pinos (confirma se correspondem à tua montagem)
#define SS_PIN 10
#define RST_PIN 9
#define DIO0_PIN 2

void setup() {
  Serial.begin(9600);
  while (!Serial); // Espera o monitor abrir

  Serial.println("\n--- INICIO DO TESTE DE HARDWARE (OLIMEX LORA868) ---");

  // 1. Configurar os pinos
  LoRa.setPins(SS_PIN, RST_PIN, DIO0_PIN);

  // 2. Tentar iniciar o módulo
  // O LoRa.begin já verifica internamente se o chip está conectado e se a versão é 0x12
  if (!LoRa.begin(868E6)) {
    Serial.println("\n ✗ ERRO: O Arduino nao consegue falar com o modulo.");
    Serial.println("  Verifica:");
    Serial.println("    - Resistencias (Divisor de tensao) nos pinos 13, 11, 10, 9.");
    Serial.println("    - Alimentacao 3.3V.");
    Serial.println("    - Se os fios estao bem apertados na breadboard.");
    while (1);
  }

  // Se passou do "if" acima, o SPI está a funcionar!
  Serial.println(" ✓ SUCESSO! O modulo foi iniciado.");
  Serial.println("  Isto confirma que as soldaduras e o divisor de tensao estao a funcionar.");

  // --- TRUQUE EXTRA ---
  // Como a funcao da biblioteca é privada, vamos ler o registo manualmente via SPI
  // só para teres o prazer de ver o "0x12" no ecrã.

  digitalWrite(SS_PIN, LOW);    // Ativa o modulo
```

```

SPI.transfer(0x42 & 0x7F);    // Envia endereço 0x42 (modo leitura)
byte version = SPI.transfer(0x00); // Lê o valor de volta
digitalWrite(SS_PIN, HIGH);    // Desativa o modulo

Serial.print("3. Versao do Chip (Lida manualmente): 0x");
Serial.println(version, HEX);

if (version == 0x12) {
    Serial.println("🐼 Confirmado: É um chip SX1276 saudavel.");
}
}

void loop() {
    // Nada a fazer
}

```

4. Plano de Desenvolvimento e Testes do Módulo de Comunicação LoRa

Este plano estabelece as próximas etapas críticas para a validação e integração do Módulo de Comunicação LoRa, em conformidade com as especificações do sistema ResQSense.

4.1. Adaptação de Software e Integração de Hardware

4.1.1. Transição da Plataforma Host (Arduino Uno → ESP32-S3)

A transição do ambiente de teste (Arduino Uno) para a plataforma de produção deve ser efetuada no ESP32-S3 WROOM-1 N8R8, que servirá como a Microcontroller Unit (MCU) Host.

- **Protocolo de Interface:** A comunicação com o Módulo LoRa868, que utiliza o chip Semtech SX1276, é realizada nativamente através da interface SPI.
- **Mapeamento de Pinos:** É mandatório redefinir as constantes SS_PIN, RST_PIN e DIO_PIN para os pinos GPIO correspondentes no ESP32-S3.
- **Compatibilidade de Tensão:** Reitera-se a atenção à diferença de voltagem (5V vs. 3.3V), embora o ESP32-S3 opere a 3.3V, a compatibilidade de E/S deve ser verificada para o módulo LoRa868, para mitigar o risco de hardware burnout.

4.2. Otimização dos Parâmetros de Rádio (Spread Factor)

O Módulo de Comunicação é vital para o sistema, devendo garantir a robustez em ambientes complexos. A otimização do Spread Factor (SF) é crucial para equilibrar a taxa de dados e o alcance.

Parâmetro	Teste de Limitação e Objetivo	KPI de Sucesso (KPI)
Spread Factor (SF)	Testar a variação do SF (e.g., SF7 a SF12) para determinar o valor que minimiza o Time On Air (ToA) , mantendo a cobertura em ambientes com obstáculos (NLOS).	O ToA deve ser o menor possível para assegurar o requisito de Latência de Alerta < 3 segundos .
Taxa de Entrega de Pacotes (PDR)	Medir a percentagem de mensagens recebidas sem perda sob diferentes distâncias e condições de obstáculo.	Perda de pacotes deve ser < 5%.
Taxa de Transmissão	O SF e a Largura de Banda (BW) devem ser compatíveis com a taxa de transmissão contínua de dados a 0.1 Hz (a cada 10 segundos).	Transmissão contínua a 0.1 Hz sem sobrecarga da rede.

4.2.1. Resultados da Otimização dos Parâmetros de Rádio (Spread Factor)

No dia 05/12/2025, foram realizados testes práticos para determinar o equilíbrio ideal entre o alcance do sinal e a latência da mensagem (Time on Air). O objetivo foi validar qual o Spreading Factor (SF) que garante a entrega do pacote dentro dos requisitos de tempo, minimizando o consumo energético. Utilizou-se um payload fixo de 17 bytes.

Tabela de Resultados:

Spreading Factor (SF)	Intervalo Médio de Recepção	Tempo no Ar Aprox. (ToA)*	RSSI Médio	SNR Médio	Observações
SF 7	~2045 ms	~45 ms	-83 dBm	9.25 dB	Muito rápido. Ideal para dados frequentes, mas menor penetração.
SF 8	~2091 ms	~91 ms	-83 dBm	11.25 dB	Aumento ligeiro na latência.
SF 9	~2169 ms	~169 ms	-97 dBm	9.00 dB	Compromisso Ideal. Latência aceitável (<200ms) e boa robustez.
SF 10	~2327 ms	~327 ms	-85 dBm	12.50 dB	Tempo no ar significativo. Útil apenas como fallback.
SF 11	~2575 ms	~575 ms	-87 dBm	11.00 dB	Alto consumo de bateria e ocupação de canal.
SF 12	~3151 ms	~1151 ms	-85 dBm	11.75 dB	Crítico: Ocupação de canal >1s. Risco elevado de colisão.

Conclusão: O Spread Factor 9 oferece um ToA de aproximadamente 170ms, permitindo uma resposta rápida a alertas críticos sem comprometer a capacidade de penetração do sinal em ambientes adversos, assim á primeira vista parece o melhor para as possíveis circunstâncias.

4.2.1.1. Códigos de Diagnóstico e Telemetria

Para validar os KPIs apresentados acima, foi desenvolvido um firmware específico para o recetor. Este código implementa o cálculo automático do intervalo entre pacotes (para detetar perdas ou atrasos) e a leitura em tempo real da qualidade do sinal (RSSI e SNR).

Código do Emissor de Diagnóstico:

```
#include <SPI.h>
#include <LoRa.h>

// --- PINOS PARA ARDUINO UNO ---
// O SPI no Uno é fixo: 11 (MOSI), 12 (MISO), 13 (SCK).
#define SS_PIN 10 // Chip Select
#define RST_PIN 9 // Reset
#define DIO0_PIN 2 // DIO0 (Interrupção - Pino 2 é obrigatório no Uno para interrupções)

// --- CONFIGURAÇÃO LORA ---
#define BAND 868E6
#define SPREADING_FACTOR 9 // SF12 (Máximo alcance, mais lento)
#define SIGNAL_BANDWIDTH 125E3
#define CODING_RATE_4 5
#define TX_POWER 20

// --- ESTRUTURA DA MENSAGEM (21 Bytes com float) ---
struct __attribute__((packed)) DataPacket {
    float lat; // 4 bytes
    float lon; // 4 bytes
    float temperatura; // 4 bytes
    float bateria; // 4 bytes
    byte statusFlag; // 1 byte
};

DataPacket myData;
unsigned long lastSendTime = 0;
int interval = 2000;

void setup() {
    Serial.begin(115200); // Confirma que o Monitor Série está a 115200
    while (!Serial);
```

```

Serial.println(F("--- Emissor Arduino Uno ---"));

// Inicializar LoRa
LoRa.setPins(SS_PIN, RST_PIN, DIO0_PIN);

if (!LoRa.begin(BAND)) {
    Serial.println(F("Falha no LoRa! Verifique ligacoes e energia (3.3V)."));
    while (1);
}

// Configurações
LoRa.setSpreadingFactor(SPREADING_FACTOR);
LoRa.setSignalBandwidth(SIGNAL_BANDWIDTH);
LoRa.setCodingRate4(CODING_RATE_4);
LoRa.setTxPower(TX_POWER);

Serial.println(F("LoRa Iniciado!"));
Serial.print(F("SF: ")); Serial.println(SPREADING_FACTOR);
}

void loop() {
    if (millis() - lastSendTime > interval) {

        // Dados fictícios
        myData.lat = 40.6412;
        myData.lon = -8.6536;
        myData.temperatura = 36.5;
        myData.bateria = 4.15;
        myData.statusFlag = 1;

        Serial.print(F("A enviar pacote... Tamanho: "));
        Serial.print(sizeof(myData));
        Serial.println(F(" bytes"));

        LoRa.beginPacket();
        LoRa.write((uint8_t*)&myData, sizeof(myData));
        LoRa.endPacket();

        lastSendTime = millis();
    }
}

```

```
}  
}
```

Código do Recetor de Diagnóstico:

```
#include <SPI.h>  
#include <LoRa.h>  
  
// --- PINOS (ARDUINO UNO) ---  
#define SS_PIN 10  
#define RST_PIN 9  
#define DIO0_PIN 2  
  
// --- CONFIGURAÇÃO LORA (IGUAL AO EMISSOR) ---  
#define BAND 868E6  
#define SPREADING_FACTOR 9  
#define SIGNAL_BANDWIDTH 125E3  
#define CODING_RATE_4 5  
  
// --- ESTRUTURA DE DADOS ---  
struct __attribute__((packed)) DataPacket {  
    float lat;  
    float lon;  
    float temperatura;  
    float bateria;  
    byte statusFlag;  
};  
  
DataPacket receivedData;  
unsigned long lastPacketTime = 0; // Para calcular o tempo entre mensagens  
  
void setup() {  
    Serial.begin(115200);  
    while (!Serial);  
  
    Serial.println("--- INICIANDO RECETOR DE DIAGNÓSTICO ---");  
  
    LoRa.setPins(SS_PIN, RST_PIN, DIO0_PIN);
```

```

if (!LoRa.begin(BAND)) {
    Serial.println("ERRO: Falha no módulo LoRa!");
    while (1);
}

// Configurações
LoRa.setSpreadingFactor(SPREADING_FACTOR);
LoRa.setSignalBandwidth(SIGNAL_BANDWIDTH);
LoRa.setCodingRate4(CODING_RATE_4);
LoRa.enableCrc();

Serial.println("Rádio Pronto. À escuta...");
Serial.println("-----");

// Inicializar o temporizador
lastPacketTime = millis();
}

void loop() {
    int packetSize = LoRa.parsePacket();

    if (packetSize) {
        // 1. Calcular tempo desde a última mensagem
        unsigned long currentTime = millis();
        unsigned long timeDiff = currentTime - lastPacketTime;
        lastPacketTime = currentTime; // Atualizar para a próxima

        // 2. Ler os dados
        if (packetSize == sizeof(DataPacket)) {
            LoRa.readBytes((uint8_t *)&receivedData, packetSize);
            printDataInfo(timeDiff);    // Função para mostrar dados
            printRadioInfo();           // Função para mostrar qualidade do sinal
        } else {
            Serial.print("ERRO: Tamanho de pacote inválido: ");
            Serial.println(packetSize);
            while (LoRa.available()) LoRa.read(); // Limpar buffer
        }
    }
}

```

```

// --- FUNÇÃO AUXILIAR: MOSTRAR DADOS ---
void printDataInfo(unsigned long timeDiff) {
    Serial.print("RX [");
    Serial.print(millis() / 1000); // Timestamp em segundos
    Serial.println("s] -----");

    Serial.print("-> Intervalo:  ");
    Serial.print(timeDiff);
    Serial.print(" ms");
    // Se demorar mais de 11s (11000ms), avisa que houve atraso/perda
    if (timeDiff > 11000) Serial.print(" [ALERTA: Atraso/Perda detetada!]);
    else if (timeDiff < 9000 && timeDiff > 0) Serial.print(" [Rápido]);
    else Serial.print(" [Normal]);
    Serial.println();

    Serial.print("-> Temperatura:  "); Serial.print(receivedData.temperatura); Serial.println(" °C");
    Serial.print("-> Bateria:      "); Serial.print(receivedData.bateria); Serial.println(" %");
    Serial.print("-> Status:      ");
    if(receivedData.statusFlag == 0) Serial.println("OK");
    else Serial.println("ALERTA / MAN DOWN");
}

// --- FUNÇÃO AUXILIAR: DIAGNÓSTICO DO RÁDIO ---
void printRadioInfo() {
    int sf = SPREADING_FACTOR;

    // Ler métricas do rádio (estas funções existem mesmo)
    int rssi = LoRa.packetRssi();
    float snr = LoRa.packetSnr();

    Serial.println("\n--- DIAGNÓSTICO DE SINAL ---");

    // 1. Mostrar SF
    Serial.print("SF Configurado: SF"); Serial.println(sf);

    // 2. Mostrar RSSI e Interpretação
    Serial.print("RSSI (Força):  "); Serial.print(rssi); Serial.print(" dBm -> ");
    Serial.println(avaliarSinal(rssi));

```

```
// 3. Mostrar SNR (Relação Sinal-Ruído)
Serial.print("SNR (Qualidade): "); Serial.print(snr); Serial.println(" dB");

Serial.println("=====\n");
}

// --- LÓGICA DE AVALIAÇÃO DO SINAL ---
String avaliarSinal(int rssi) {
  if (rssi > -90) return "EXCELENTE (Perto)";
  if (rssi > -105) return "BOM (Estável)";
  if (rssi > -120) return "FRACO (Limite da ligação)";
  return "CRÍTICO (Perda iminente)";
}
```

4.3. Definição da Formatação da Mensagem

Para garantir a eficiência energética e cumprir o requisito de transmissão a cada 10 segundos (0.1 Hz) sem saturar a rede LoRa, o sistema utiliza serialização binária direta em vez de formatos de texto (como JSON ou CSV).

Esta abordagem reduz o tamanho do pacote para um valor fixo de 17 Bytes, diminuindo significativamente o Time on Air. Isto permite a utilização de Spreading Factors mais elevados (como o SF9 validado nos testes), essenciais para garantir o alcance em ambientes com obstáculos, sem comprometer a autonomia da bateria.

4.3.1. Conteúdo do Payload e Estrutura

O pacote deve agregar os seguintes dados provenientes dos módulos:

- **Identificação:** Identifier (ID) do Operacional.
- **Localização:** Posição absoluta (GNSS) e relativa (UWB/IMU).
- **Biométricos:** Frequência Cardíaca (BPM), Níveis de SpO2 e Temperatura Corporal.
- **Estado:** Detecção do estado físico (e.g., “fallen”, “ok”) e Alertas de Alta Prioridade (e.g., 'Man Down', SpO2 Crítico).

4.3.2. Estrutura da mensagem para teste do SpreadFactor no arduino

A mensagem é estruturada num único pacote de dados (struct), compactado para evitar desperdício de memória (padding). A estrutura garante a compatibilidade entre o transmissor (Arduino Uno / 8-bit) e o futuro recetor (ESP32 / 32-bit).

Byte(s)	Variável	Tipo de Dados	Tamanho	Descrição
0-3	lat	float (IEEE 754)	4 Bytes	Latitude GNSS (precisão aprox. 1m)
4-7	lon	float	4 Bytes	Longitude GNSS
8-11	temperatura	float	4 Bytes	Temperatura corporal do operacional (°C)
12-15	bateria	float	4 Bytes	Nível de tensão da bateria (%)
16	statusFlag	byte (uint8_t)	1 Byte	Estado: 0=OK, 1=Man Down, 2=SOS
TOTAL			17 Bytes	

4.3.3. Implementação de Segurança e Integridade

A estrutura da mensagem final deve incorporar os seguintes mecanismos de segurança, antes de ser enviada ao transmissor LoRa:

1. **Encriptação:** O payload deve ser encriptado usando o algoritmo AES-128-CTR.
2. **Integridade:** O pacote deve incluir um CRC (Cyclic Redundancy Check) para permitir ao recetor a deteção de erros durante a transmissão.

Nota: A definição de um formato binário ou de um formato de texto compacto (e.g., JSON otimizado com chaves curtas) deve ser priorizada para maximizar a eficiência do payload.

5. Bibliografia

Documentação LoRa:

(<https://randomnerdtutorials.com/esp32-lora-rfm95-transceiver-arduino-ide/>)

<https://github.com/OLIMEX/LoRa-868-915/tree/main/SOFTWARE/Arduino>

<https://www.makerhero.com/blog/comunicacao-lora-com-arduino-p2p/>

Documentação importante do ESP:

<https://www.botnroll.com/pt/esp32/5227-esp32-s3-placa-da-waveshare-com-esp32-s3-wroom-1-n8r8-240mhz-dual-core-usb-c-headers-soldados.html>

<https://www.waveshare.com/wiki/ESP32-S3-DEV-KIT-N8R8>

6. Histórico de versões

Versão	Data	Descrição das Alterações
1.0	01/12/2025	Versão inicial da documentação LoRa.
1.1	02/12/2025	Adição do esquema da interface LoRa -> Arduino e adição do código teste de cada módulo individual.
1.2	05/12/2025	Inclusão dos resultados dos testes práticos de otimização de Spread Factor (SF), análise de latência (Time on Air) e melhor definição da estrutura da mensagem (Payload). Adição do código do Emissor e do Recetor com Diagnóstico e Telemetria.